<table>
<tr><td colspan="2">Practical implementation of Scanner</td></tr>
<tr><td>Step 1</td><td>Generate text file for given Input</td></tr>
<tr><td>Step 2</td><td>Declare two static table for Operator and Keywords</td></tr>
<tr><td>Step 3</td><td>Declare two dynamic table for constant and Symbol</td></tr>
<tr><td>Step 4</td><td>Read Input file apply STRTOK () to tokenize given input string (get logic from Help menu)</td></tr>
<tr><td>Step 5</td><td>In tokenization While loop, for each token<ul><li>Check for keywords from keyword table if it exists then print [KW#index], where index is the record number in the respective table.</li><li>Else Check for Operator from Operator table if it exists then print [OP#index], where index is the record number in the respective table.</li><li>Else check that given token is digit then check whether it exists in constant table then print [CO#index], where index is the record number in the respective table, else store digits in constant table then print [CO#index]</li><li>Else check that given token exists in symbol table then print [ID#index], where index is the record number in the respective table, else store symbol in symbol table then print [ID#index]</li></ul></td></tr>
<tr><td>Ex:</td><td>INPUT :<br>INT a , b ;<br>REAL c , d ;<br>a = b + c * 100 ;<br>d = a – 90 ;<br><br>Static Table:<table><tr><td>OP TABLE</td><td>,</td><td>;</td><td>=</td><td>*</td><td>+</td><td>-</td></tr></table><table><tr><td>KW TABLE</td><td>INT</td><td>REAL</td></tr></table>Dynamic table:<table><tr><td>ID TABLE</td><td>a</td><td>b</td><td>c</td><td>d</td></tr></table><table><tr><td>CO TABLE</td><td>100</td><td>90</td></tr></table>OUTPUT:<br><br>[KW #1] [ID #1] [OP #1] [ID#2][OP#2]<br>[KW#2] [ID#3][OP#1][ID#4][OP#2]<br>[ID#1][OP#3][[ID#2][OP#5][ID#3][OP#4][CO#1][OP#2]<br>[ID#4][OP#3][ID#1][OP#6][CO#2][OP#2]</td></tr>
<tr><td></td><td></td></tr>
</table>

## Code:-

```
#include <stdio.h>
```

```
#include <string.h>
#include <ctype.h>
#include <conio.h> // TurboC specific header for console I/O

char kw[32][10] = {"int", "float", "while", "for", "do", "char", "break",
                   "auto", "continue", "default", "double", "if", "else",
                   "enum", "goto", "long", "switch", "typedef", "union",
                   "unsigned", "void", "volatile", "extern", "case",
                   "const", "return", "sizeof", "static", "struct",
                   "register", "signed"};

char op[15] = {'+', '-', '*', '/', '=', ':', ';', '<', '>',','};

char identifiers[20][10]; // Global array to store identifiers
char constants[20][10];   // Global array to store constants
int ic = 0, cc = 0;       // Global counters for identifiers and constants

void analyzeString(char str[]);

int main() {
    FILE *file;
    char str[100];

    file = fopen("input.txt", "r");

    if (file == NULL) {
        printf("Error opening the file.\n");
```

```
        getch(); // Wait for a key press
        return 1; // Return an error code
    }

    while (fgets(str, sizeof(str), file) != NULL) {
        analyzeString(str);
    }

    fclose(file);

    getch(); // Wait for a key press before closing the console window
    return 0;
}

void analyzeString(char str[]) {
    char *ptr;
    int i, j;

    ptr = strtok(str, " \n");

    while (ptr != NULL) {
        int flag = 0;

        for (i = 0; i < 32; i++) {
            if (strcmp(ptr, kw[i]) == 0) {
                printf("KW#%d ", i + 1);
                flag = 1;
```

```
            break;
        }
    }

    if (flag == 0) {
        for (j = 0; j < 10; j++) {
            if (ptr[0] == op[j]) {
                printf("OP#%d ", j + 1);
                flag = 1;
                break;
            }
        }
    }

    if (flag == 0) {
        if (isalpha(ptr[0])) {
            int isRepeated = 0;
            for (i = 0; i < ic; i++) {
                if (strcmp(ptr, identifiers[i]) == 0) {
                    printf("ID#%d ", i + 1);
                    isRepeated = 1;
                    break;
                }
            }
            if (!isRepeated) {
                strcpy(identifiers[ic++], ptr);
                printf("ID#%d ", ic);
```

```
                }
            } else if (isdigit(ptr[0])) {
                int isRepeated = 0;
                for (i = 0; i < cc; i++) {
                    if (strcmp(ptr, constants[i]) == 0) {
                        printf("CO#%d ", i + 1);
                        isRepeated = 1;
                        break;
                    }
                }
                if (!isRepeated) {
                    strcpy(constants[cc++], ptr);
                    printf("CO#%d ", cc);
                }
            }
        }

        ptr = strtok(NULL, " \n");
    }
}
```

**File—**

```
int a , b ;
float c , d ;
a = b + c * 100 ;
d = a – 90 ;
c = 90 ;
```